

13.4.2 对象持久化与数据库

在面向对象开发方法中，对象只能存在于内存中，而内存不能永久保存数据，如果要永久保存对象的状态，需要进行对象的持久化（persistence），对象持久化是把内存中的对象保存到数据库或可永久保存的存储设备中。在多层软件设计和开发中，为了降低系统的耦合度，一般会引入持久层（Persistence Layer），即专注于实现数据持久化应用领域的某个特定系统的一个逻辑层面，将数据用户和数据实体相关联，持久层的设计实现了数据处理层内部的业务逻辑和数据逻辑的解耦。

目前，关系数据库是使用最为广泛的数据库，如 DB2、Oracle、SQL Server 等。关系数据库中存放的是关系数据，即用二维表格表示的数据，它是非面向对象的。对象和关系数据其实是业务实体的两种表现形式，业务实体在内存中表现为对象，在数据库中表现为关系数据。内存中的对象之间存在关联和继承关系，而在数据库中，关系数据无法直接表达多对多关联和继承关系。因此，将对象持久化到关系数据库中，需要进行对象 / 关系的映射（Object/Relation Mapping，ORM），这是一项非常重要且烦琐耗时的工作。

在实际应用中，除了需要将内存中的对象持久化到数据库外，还需要将数据库中的关系数据再重新加载到内存中，以满足用户查询业务数据的需求。频繁地访问数据库，会对应用的性能造成很大影响，为了降低访问数据库的频率，可以将需要经常被访问的业务数据存放在缓存中，并且通过特定的机制来保证缓存中的数据与数据库中的数据同步。

数据持久化技术封装了数据访问细节，为大部分业务逻辑提供面向对象的 API。通过持久化技术，可以减少访问数据库数据的次数，增加应用程序执行速度；其代码重用性高，能够完成大部分数据库操作；松散耦合，使持久化不依赖于底层数据库和上层业务逻辑实现，更换数据库时只需修改配置文件而不用修改代码。随着对象持久化技术的发展，诞生了越来越多的持久化框架，目前，主流的持久化技术框架包括 CMP、Hibernate、iBatis 和 JDO 等。

1. CMP

在 J2EE 架构中，容器管理持久化（Container-Managed Persistence，CMP）是由 EJB 容器来管理实体 EJB 的持久化，EJB 容器封装了对象 / 关系的映射和数据访问细节。CMP 和 ORM 的相似之处在于，两者都提供对象 / 关系映射服务，都将对象持久化的任务从业务逻辑中分离出来，区别在于 CMP 负责持久化实体 EJB 组件，而 ORM 负责持久化简单的 Java 对象（Plain Ordinary Java Object，POJO），它是普通的基于 Java Bean 形式的实体域对象。

CMP 模式的优点在于它基于 EJB 技术，是 SUN J2EE 体系的核心部分，获得了业界的普遍支持，包括各大厂商和开源组织等。如果选择它来进行企业级开发，技术支持会非常完备，同时其功能日趋完善，包括了完善的事务支持、EJBQL 查询语言和透明的分布式访问等。CMP 的缺点在于开发的实体必须遵守复杂的 J2EE 规范，而 ORM 没有类似要求，其灵活性受到影响；而且，CMP 只能运行在 EJB 容器中，而普通 POJO 可以运行在任何一种 Java 环境中；尽管遵循 J2EE 的规范，但 EJB 的移植性比 ORM 要差。